

TUTORIAL – IMPLEMENTANDO UMA PILHA DE AÇÕES EM PYTHON

Objetivo do tutorial

Ao final deste tutorial, você terá construído um programa em Python que:

- Utiliza a estrutura de dados **Pilha**
 - Segue o conceito **LIFO (Último a entrar, primeiro a sair)**
 - Usa **funções** para organizar o código
 - Simula o funcionamento do botão **Desfazer (Undo)**
-

ETAPA 1 – ENTENDENDO O PROBLEMA

Antes de programar, precisamos entender o que o sistema deve fazer.

O programa irá:

- Guardar ações realizadas pelo usuário
- Sempre armazenar a ação mais recente no topo da pilha
- Permitir desfazer apenas a última ação feita

Isso significa que:

- Não podemos remover qualquer ação aleatória
- Sempre removemos **a última ação inserida**

Esse comportamento é exatamente o de uma **pilha**.

ETAPA 2 – CRIANDO A PILHA

Em Python, uma pilha pode ser representada por uma **lista**.

O que você deve fazer

Crie uma lista vazia que irá armazenar as ações do usuário.

Por que isso funciona

- Listas permitem inserir e remover elementos do final
- O final da lista representará o **topo da pilha**

Código esperado nesta etapa

```
acoes = []
```

Neste momento:

- A pilha está vazia
 - Nenhuma ação foi registrada ainda
-

ETAPA 3 – CRIANDO A FUNÇÃO PARA ADICIONAR UMA AÇÃO (PUSH)

Agora vamos criar uma função responsável por:

- Pedir ao usuário uma ação
- Inserir essa ação na pilha

O que essa função deve fazer

1. Pedir ao usuário que descreva a ação
2. Inserir essa ação no topo da pilha
3. Informar que a ação foi adicionada com sucesso

Por que usar função

- Evita repetição de código
- Torna o programa mais organizado
- Facilita manutenção e leitura

Estrutura da função

```
def adicionar_acao(pilha): acao = input("Digite a ação realizada: ")
    pilha.append(acao) print("Ação adicionada com sucesso.")
```

Conceito aplicado

- `append()` adiciona um elemento ao final da lista
- O final da lista representa o topo da pilha
- Isso caracteriza a operação **push**

ETAPA 4 – CRIANDO A FUNÇÃO PARA DESFAZER AÇÃO (POP)

Agora vamos permitir que o usuário desfça a última ação realizada.

Problema importante

Não é possível remover uma ação se a pilha estiver vazia.

Portanto, **antes de remover**, precisamos verificar:

- A pilha tem elementos?

O que a função deve fazer

1. Verificar se a pilha está vazia
2. Se estiver vazia, mostrar uma mensagem de aviso
3. Se não estiver vazia, remover a última ação
4. Informar qual ação foi desfeita

Estrutura da função

```
def desfazer_acao(pilha): if len(pilha) == 0: print("Nenhuma ação para desfazer.") else: acao_removida = pilha.pop() print(f"Ação desfeita: {acao_removida}")
```

```
{acao_removida}")
```

Conceito aplicado

- `pop()` remove o último elemento da lista
 - Isso representa a operação **pop** da pilha
 - O comportamento segue LIFO
-

ETAPA 5 – CRIANDO A FUNÇÃO PARA MOSTRAR AS AÇÕES

Agora vamos permitir que o usuário visualize o conteúdo da pilha.

O que a função deve fazer

1. Verificar se a pilha está vazia
2. Se estiver vazia, informar ao usuário
3. Se não estiver vazia:
 - Mostrar todas as ações
 - Indicar qual é a ação do topo

Estrutura da função

```
def mostrar_acoes(pilha): if len(pilha) == 0: print("Nenhuma ação registrada.")  
else: print("\nAções realizadas:") for acao in pilha: print("-", acao)  
print(f"Topo da pilha: {pilha[-1]}")
```

Conceitos importantes

- `pilha[-1]` acessa o último elemento da lista
 - Esse elemento representa o topo da pilha
-

ETAPA 6 – CRIANDO O MENU DO SISTEMA

Agora vamos criar o menu que permite ao usuário interagir com o sistema.

O que o menu deve fazer

- Mostrar opções
- Ler a escolha do usuário
- Retornar a opção escolhida

Estrutura do menu

```
def mostrar_menu(): print("\n===== MENU =====") print("1 - Adicionar ação")  
print("2 - Desfazer ação") print("3 - Mostrar ações") print("4 - Sair") return  
input("Escolha uma opção: ")
```

ETAPA 7 – CRIANDO O PROGRAMA PRINCIPAL

Agora vamos unir todas as partes.

O que o programa principal deve fazer

- Repetir o menu até o usuário escolher sair
- Chamar a função correta conforme a opção escolhida

Estrutura do loop principal

```
while True: opcao = mostrar_menu() if opcao == "1": adicionar_acao(acoes) elif  
opcao == "2": desfazer_acao(acoes) elif opcao == "3": mostrar_acoes(acoes) elif  
opcao == "4": print("Encerrando o sistema.") break else: print("Opção  
inválida.")
```

ETAPA 8 – TESTANDO O SISTEMA

O aluno deve testar:

- Adicionar várias ações
 - Desfazer mais de uma ação
 - Tentar desfazer com a pilha vazia
 - Verificar se o topo da pilha está correto
-

O QUE FOI APRENDIDO NESTA ATIVIDADE

Ao concluir este tutorial, você trabalhou:

- Estrutura de dados **Pilha**
- Conceito **LIFO**
- Uso de listas como pilha
- Modularização com funções
- Organização de um programa interativo